

Team Project SWE Lab

Fall 2026

Important Links

Project Guidelines: <https://utexas.instructure.com/courses/1455613/files?preview=92989463>

Git repository: <https://github.com/shreyashetty-24/Swe26>

User Stories: <https://www.mountaingoaftware.com/agile/user-stories#what-is-a-user-story>

Project Description

HaaS System Project Description Create a PoC for a web application for a functioning HaaS system to enable users to achieve the following Stakeholder Needs (SN).

- SN0: Accepted quality and reliability metrics
- SN1: Create and maintain secure user accounts and projects on the system
- SN2: View the status of all hardware resources in the system
- SN3: Request available hardware resources S
- N4: Checkout and manage these resources
- SN5: Check-in the resources and get status of all hardware resources in the system
- SN6: Deliver PoC within schedule constraints, with support for scalability

Based on these stakeholder needs, your team will define system requirements. The table below lists some of the system requirements to help you get started. You can use these requirements as a starting point and further refine them and add more. As discussed in the class, you should map each requirement to one or more stakeholder needs. Likewise, you should ensure that all stakeholder needs are met.

Rubric For Checkpoint 1

R1-1 (1 points)

Project Plan:

- Team members: Nina, Ify, Nicole, Ishanvi, and Shreya
- Sprint cadence: Once a week (as in weekly check ins, not weekly work)

- Collaboration tools: Git, Google Docs, Zoom
- Implementation methodology: Agile
- Toolchain: VSCode, Github

R1-2 (3 points)

All Features on a board.

Initial work items created for features.

- User stories
-
- Technical debt
- Research items

User Management, Project Management, Hardware Management

Users are in projects, projects can checkout hardware

Hardware sets are high-end, returnable baking tools (e.g., Stand Mixers, Espresso Machines, Proofing Cabinets, Commercial Dehydrators). Projects are **baking recipes/orders** (e.g., "Wedding Cake Project", "Artisan Sourdough Batch").

- **User (usersDB.py):** Bakers, Pastry Chefs, or Baristas.
- **Project (projectsDB.py):** Specific baking orders or seasonal menu items requiring dedicated tools for a specific time window.
- **Hardware Set (hardwareDB.py):** Shared equipment pools with set total capacities (e.g., **HardwareSet1:** "KitchenAid 8Qt Mixers", Capacity: 10).
- **Check-Out / Check-In:** A chef reserves 2 stand mixers to complete "Wedding Cake Project", then returns them to the shared kitchen pool when finished.

Who are users → Employees

What are the projects → Pastry

What is the hardware → Equipment (mixers, spoons, whisks, piping tips)

- Possible extra feature is keeping track of ingredient inventory

-login, join projects, select type and quantity of equipment, time frame of use, checkout, check in, have an inventory/see what we have, know if there isn't enough

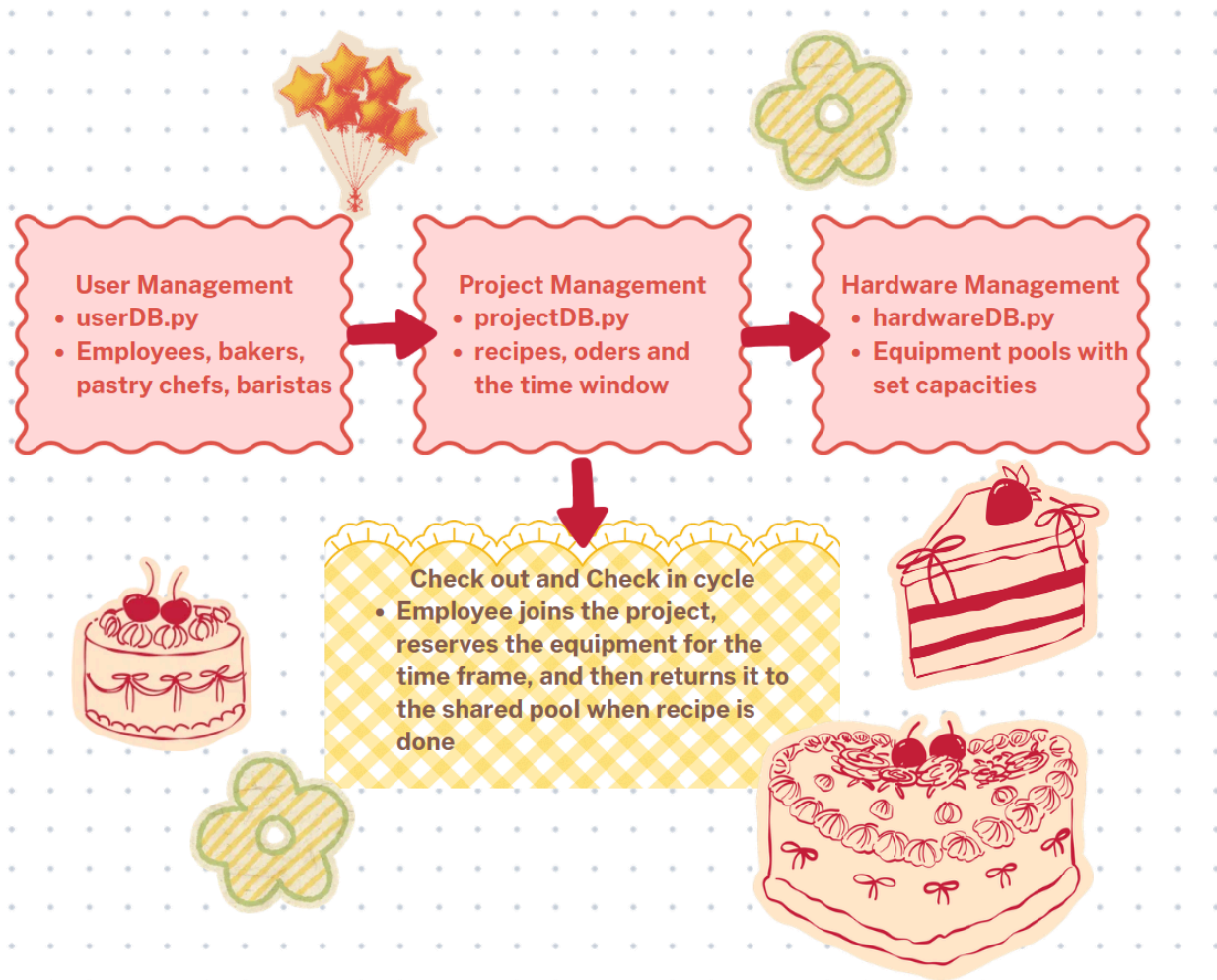
User Stories:

As a [type of user], I [need/want/am required] to [do something], so that [reason or benefit].

1. As an Employee, I need to login, so that I can access the list of recipes.

2. As an Employee, I need to join an existing project, so that I can share equipment access with teammates working the same order.
 3. As an Employee, I need to set a time window for my project, so that equipment is reserved only for as long as I need it.
 4. As a Pastry recipe, I need to check out a resources, so that I can make a pastry
 - a. Maybe later we can specify which team member checked it out
 5. As a Pastry recipe, I need to return a resource, so that others can use it
 6. As an employee, I need to make a new login(username/password), in order to login
 7. As an employee, I need to join a pastry recipe, so that I can access the resource log/checkout.
 8. As an Employee, I need to see an error/warning if requested hardware isn't available, so that I don't over-commit my project timeline.
 9. As an Employee, I need to view all projects I'm part of, so that I can track my active work.
 10. As an admin/system, I need to log which employee checked out which item, so that accountability is maintained.
-
- Research: evaluate Flask vs FastAPI for the PoC backend
 - Research: authentication approach (plaintext vs hashed passwords — needed even for PoC, per SN1 "secure")
 - Research: concurrency handling for simultaneous checkout requests on limited-capacity hardware
 - Tech debt (anticipated): no admin UI in PoC — manual DB seeding for hardware sets initially
 - Tech debt (anticipated): no ingredient-inventory tracking in v1 (flagged as stretch feature)
 - Research: how to demo scalability (SN6) meaningfully in a PoC — load testing approach or just architectural discussion?

R1-3 (1 point)



High level sketch (on paper, PowerPoint, draw.io, Visio) of the Application

User (usersDB.py) → Employees (Bakers/Pastry Chefs/Baristas); handles auth, account creation (SN1)

Project (projectsDB.py) → Pastry recipes/orders with a defined time window; users join projects (SN1, SN3)

Hardware Set (hardwareDB.py) → Equipment pools with fixed capacity (mixers, spoons, whisks, piping tips); tracks availability (SN2, SN3, SN4, SN5)

R1-4 (1 point)

Language: Python backend (given usersDB.py/projectsDB.py/hardwareDB.py naming), with a lightweight web framework (Flask or FastAPI) suggested for the PoC, simple to stand up quickly, good for scalability discussion (SN6)

Version control: GitHub, supports branching per feature, PR review, issue/board tracking in one place (also covers SN6 schedule constraints via visibility)

Board/tracking: GitHub Projects (keeps user stories, tech debt, and research items in the same ecosystem as code, avoids tool sprawl)

Communication: Zoom for weekly sync, Google Docs for living design docs (SNO quality/reliability discussion, requirements traceability matrix)

Rationale: Agile fits because stakeholder needs are still loosely defined and requirements will get refined sprint-over-sprint; weekly cadence matches a class schedule better than daily stand-ups for a 5-person part-time team